Algorithms

Lecture 5

Karatsuba

by Zarin Tasnim Biash

Karatsuba [Simulation]

$x_n , x_{n-1} , \ldots\ldots\ldots , x_1 ;$ $y_n , y_{n-1} , \ldots\ldots\ldots , y_1$

$x_n , x_{n-1} , \ldots\ldots\ldots , x_{n/2+1} , x_{n/2} , x_{n/2-1} , \ldots\ldots\ldots , x_1$

xH xL

Let's take x = 1234 and y = 5678.

Comparing this with the previous example, xH = 12 and xL = 34

$y_n , y_{n-1} , \ldots\ldots\ldots , y_{n/2+1} , y_{n/2} , y_{n/2-1} , \ldots\ldots\ldots , y_1$

yH yL

Comparing this with the previous example, yH = 56 and yL = 78

Karatsuba [Simulation]

$x = x_H * 10^{n/2} + x_L$; $y = y_H * 10^{n/2} + y_L$

$xy = (x_H . y_H)10^n + (x_H . y_L + x_L . y_H)10^{n/2} + x_L . y_L$

$x_H . y_H = a$

$x_L . y_L = b$

a and b are both multiplications of two n/2 bit numbers.

$x_H . y_L + x_L . y_H = (x_H + x_L)(y_H + y_L) - x_H . y_H - x_L . y_L$

$(x_H + x_L)(y_H + y_L) = c$

c is the multiplication of two n/2 or (n/2 + 1) bit numbers

$xy = a.10^n + (c-a-b)10^{n/2} + b$

Karatsuba [Simulation]

Let's take 1234 x 5678 as an example.

= (12 x 10**2 + 34) (56 x 10**2 + 78) :

= (12 x 56) 10**4 + (12 x 78) 10**2 + (34 x 56) 10**2 + (34 x 78) :

$x_H . y_H$ = a = (12 x 56) $x_L . y_L$ = b = (34 x 78)

a and b are both multiplications of two n/2 bit numbers.

(12 x 78) + (34 x 56) = (12 + 34)(78 + 56) - (12 x 56) - (34 x 78)
c - a - b

(12 + 78)(34 + 56) = c

c is the multiplication of two n//2 or (n//2 + 1) bit numbers

Ergo, xy = (12 x 56). 10**4 + ((12+78)(34+56) - (12 x 56) - (34 x 78)). 10**2 + (34 x 78)

$x = x_H * 10^{n/2} + x_L$; $y = y_H * 10^{n/2} + y_L$

$xy = (x_H . y_H)10^n + (x_H . y_L + x_L . y_H)10^{n/2} + x_L . y_L$

$x_H . y_L + x_L . y_H = (x_H + x_L)(y_H + y_L) - x_H . y_H - x_L . y_L$

$(x_H + x_L)(y_H + y_L) = c$

$xy = a.10^n + (c-a-b)10^{n/2} + b$

Karatsuba [Pseudo Code]

```
def karatsuba (x, y):
        if len(str(x)) == 1 or len(str(y))==1:
                return x * y
        else:
                n = max(len(str(x)), len(str(y)))     #in main code, adjust the length if unequal
                m = ceil(n/2) or floor(n/2)
                p = 10 ** m
                xh = x//p
                xl = x%p
                yh = y//p
                yl = y%p
                a = karatsuba(xh, yh)
                b = karatsuba(xl, yl)
                c = karatsuba(xh+xl, yh+yl)
                return (a*(10**n) + (c-a-b)*(10**m) + b)
```

Karatsuba [Time Complexity]

$T(n) = 3T(n/2) + O(n)$

$O(n)$ => To add or subtract 2 n-bit numbers.

We will look into this in more details when we will cover Master's theorem.

$T(n) = O(n^{\log 3}) \approx O(n^{1.585})$ which is less than $O(n^2)$

N.B. Karatsuba works for any base (not only 10)

Karatsuba [Time Complexity] continued...

Why Karatsuba? Why not normal multiplication?

Because normal multiplication does n^2 single digit multiplications, which leads the complexity to be $O(n^2)$.

But Karatsuba makes it much better. It actually brings the complexity down to $O(n^{log3})$ which is almost $O(n^{1.585})$.

But how?

Karatsuba [Example]

Let’s look at this first:

1234 x 5678

$= (12 \times 10^2 + 34)(56 \times 10^2 + 78)$

$= (12 \times 56)\ 10^4 + (12 \times 78 + 34 \times 56)\ 10^2 + (34 \times 78)$

Here we can see that, we can calculate this multiplication of two 4-digit numbers, using 4 multiplications of 2-digit numbers.

But we can actually do this using 3 multiplications of 2-digit numbers, instead of 4 multiplications.

Karatsuba [Example]

The term (12 x 78 + 34 x 56) can also be written as:

= (12 + 34) (56 + 78) - (12 x 56) - (34 x 78)

Here, we have already computed (12 x 56) and (34 x 78). So we only need to compute (12 + 34) (56 + 78) now, which is a single multiplication of two 2 or 3 digit numbers.

In short, calculating (12 + 34) (56 + 78) needs a single multiplication, whereas calculating (12 x 78 + 34 x 56) takes two multiplications.

Karatsuba [Example]

If we think about multiplying two n-digit numbers, then it becomes like this:

Instead of multiplying two n/2 digit numbers 4 times, we can calculate the same result by multiplying two n/2 digit numbers 3 times.

If we didn't make this improvement, then using 4 multiplications would lead the complexity to be -> $O(n^{log4}) = O(n^2)$, which is the same as the normal multiplication.

But using 3 multiplications make it $O(n^{log3})$ or almost $O(n^{1.585})$.